

AI Camera — อ่านป้ายทะเบียนด้วย Raspberry Pi 5

อบรมเชิงปฏิบัติการ 2 วัน สำหรับนักเรียนระดับมัธยมปลาย

ระยะเวลา

2 วัน (09:00–16:30)

HARDWARE หลัก

Raspberry Pi 5 + Camera Module 3

เทคโนโลยี

Python · OpenCV · YOLOv8n · Tesseract OCR



กล้อง



YOLOv8n



Crop



OCR



Whitelist



แสดงผล

วันที่ 1



พื้นฐานคอมพิวเตอร์ + Python + Hardware + Image Processing

🎯 พื้นฐาน RPi + Python + pipeline การมองภาพแบบคลาสสิก

เปิดคอร์ส + แนะนำโปรเจกต์

- 🎬 **Demo ระบบอ่านป้ายทะเบียนที่ทำงานจริง** — เห็นเป้า
หมายก่อนเริ่มเรียน
- ภาพรวม 2 วัน: จากศูนย์ → ระบบที่ตรวจจับป้าย อ่านข้อความ
และเทียบ **whitelist** ได้เอง
- แนะนำตัวผู้สอน ทีมงาน และกติกาห้องเรียน
- ภาพรวม timeline: เข้าปูพื้นฐาน บ่ายลงมือทำจริง



[ใส่วิดีโอ/ภาพ demo จริงของ Cytron]

แนะนำใช้คลิป demo ระบบ ANPR ที่ Cytron เคยทำจริง
แทน stock image

ความเป็นมาของ Raspberry Pi เทียบกับ PC

- Raspberry Pi ถือกำเนิดปี 2012 ที่อังกฤษ เพื่อให้เด็กเข้าถึงคอมพิวเตอร์ราคาถูกลง
- **SBC (Single Board Computer)** รวมทุกอย่างไว้บนบอร์ดเดียว ต่างจาก PC ที่แยกชิ้นส่วน
- เทียบขนาด/ราคา/พลังงาน — Pi เล็กเท่าบัตรเครดิต กินไฟน้อยกว่า PC มาก
- Use case ที่ SBC เหมาะกว่า PC: **หุ่นยนต์ IoT embedded system** ที่ต้องพกพา/ฝังตัว

 Raspberry Pi 5 board

Raspberry Pi 5 — ภาพจาก Wikimedia Commons (แนะนำเปลี่ยนเป็นภาพสินค้าของ Cytron เอง)

ส่วนประกอบพื้นฐานของคอมพิวเตอร์

- **CPU** — หน่วยประมวลผลกลาง คำนวณและตัดสินใจทุกคำสั่ง
- **RAM** — ความจำชั่วคราวระหว่างโปรแกรมทำงาน ยิ่งเยอะยิ่งรันได้ลื่น
- **Storage (microSD)** — พื้นที่เก็บ OS และไฟล์แบบถาวร
- **GPU** — ประมวลผลภาพ/กราฟิก และช่วยเร่งงาน AI บางส่วน
- **I/O** — พอร์ตเชื่อมต่อ USB, HDMI, GPIO, CSI (สำหรับกล้อง)



Raspberry Pi 5 board layout

[แนะนำแปะ label CPU/RAM/USB/GPIO กับภาพบอร์ดจริงก่อนสอน]

รู้จัก Raspberry Pi 5 ภาคปฏิบัติ

- เปิดเครื่องครั้งแรก ต้องจอ คีย์บอร์ด เมาส์ เข้าสู่ **Raspberry Pi OS**
- สำรวจหน้าต่าง Desktop และเมนูตั้งค่าพื้นฐาน
- เปิด **Terminal** ลองคำสั่งพื้นฐาน: `ls`, `cd`, `pwd`, `python3 --version`
- Terminal คือเครื่องมือหลักที่จะใช้ตลอด 2 วันนี้

```
Terminal

pi@raspberrypi:~ $ ls

Desktop  Documents  Downloads

pi@raspberrypi:~ $ pwd

/home/pi

pi@raspberrypi:~ $ python3 --version

Python 3.11.2
```

Python คืออะไร

- ภาษาโปรแกรมที่อ่านง่าย เขียนสั้น ใกล้เคียงภาษาอังกฤษปกติ
- **Code** คือชุดคำสั่งที่บอกคอมพิวเตอร์ว่าให้ทำอะไร ทีละขั้นตอน
- คอมพิวเตอร์ทำงานจากบนลงล่าง ทีละบรรทัด ตามลำดับที่เราเขียน
- เป็นภาษาที่ใช้กว้างที่สุดในสาย AI/Data/Robotics — รวมถึงสิ่งที่เราจะสร้างใน 2 วันนี้

```
hello.py

# โปรแกรม Python บรรทัดแรกของเรา

print("Hello, Cytron!")

# ผลลัพธ์ที่หน้าจอ

Hello, Cytron!
```

ตัวแปร (Variable) และชนิดข้อมูล (Data Type)

- ตัวแปร คือ "กล่อง" ที่มีชื่อ ใช้เก็บค่าไว้เรียกใช้ภายหลัง
- ตั้งชื่อกล่องเอง แล้วใส่ค่าด้วยเครื่องหมาย = เช่น **name** = "Got"
- ชนิดข้อมูลหลัก: **int** (จำนวนเต็ม), **float** (ทศนิยม), **str** (ข้อความ), **bool** (True/False)
- Python รู้ชนิดข้อมูลให้เองอัตโนมัติ ไม่ต้องประกาศชนิดล่วงหน้า

```
variables.py

age = 16          # int

height = 168.5    # float

name = "Got"      # str

is_student = True # bool
```


print() และ input() — ให้โปรแกรมคุยกับเรา

- **print()** ใช้แสดงผลหรือออกทางหน้าจอ
- **input()** ใช้รับค่าที่ผู้ใช้พิมพ์เข้ามา — ได้ผลลัพธ์เป็น **str** เสมอ
- นำค่าที่รับมาไปใช้ต่อได้ทันที เช่น ทักทายกลับด้วยชื่อที่พิมพ์เข้ามา

```
greet.py

name = input("ชื่อของคุณ: ")

print("สวัสดี", name)

# ผู้ใช้พิมพ์: Got

# ผลลัพธ์: สวัสดี Got
```

Operators — คำนวณและเปรียบเทียบ

- Operator คำนวณ: + - * / และ // (หารปัดเศษ), % (หารเอาเศษ)
- Operator เปรียบเทียบ: == != > < >= <= ได้ผลลัพธ์เป็น True/False
- ผลจากการเปรียบเทียบ ใช้ไปตัดสินใจต่อในขั้น **if-else**

```
operators.py

5 + 3      # 8

10 % 3     # 1   ( เศษ )

5 > 3      # True
```

เงื่อนไข if-else

- เหมือนทางแยกถนน — ต้องเลือกทางใดทางหนึ่งตามเงื่อนไข
- **if** ตรวจสอบเงื่อนไข ถ้าเป็นจริง ให้ทำคำสั่งข้างใน
- **else** จัดการกรณีที่เงื่อนไขไม่จริง
- **elif** ใช้เมื่อมีหลายเงื่อนไขต่อกัน

```
grade.py

if score >= 50:

    print("ผ่าน")

else:

    print("ไม่ผ่าน")
```

loop (for / while) — ทำซ้ำโดยไม่ต้องเขียนซ้ำ

- เหมือนสิ่ง"วิ่งรอบสนาม 3 รอบ" — บอกครั้งเดียว ไม่ต้องพูดซ้ำ 3 ครั้ง
- **for** ใช้ทำซ้ำตามจำนวนรอบ/รายการที่กำหนดไว้ล่วงหน้า
- **while** ใช้ทำซ้ำ "จนกว่า" เงื่อนไขจะเป็นเท็จ
- ⚠ ระวัง **infinite loop** — ลืมอัปเดตเงื่อนไขจนวนไม่รู้จบ

```
loop.py

for i in range(3):

    print("รอบที่", i)

# รอบที่ 0 / รอบที่ 1 / รอบที่ 2
```

list — เก็บข้อมูลหลายค่าไว้ในตัวแปรเดียว

- เหมือนตะกร้าใส่ของหลายชิ้น เรียงเป็นลำดับในที่เดียว
- เข้าถึงแต่ละค่าด้วย **index** โดยเริ่มนับที่ **0**
- เพิ่ม/ลบ/แก้ไขค่าในลิสต์ได้ตลอดเวลา
- ใช้คู่กับ **loop** เพื่อวนดูข้อมูลที่ละตัวได้สะดวก

```
fruits.py

fruits = ["แอปเปิ้ล", "กล้วย", "ส้ม"]

print(fruits[0]) # แอปเปิ้ล

for f in fruits:

    print(f)
```

function — เก็บคำสั่งเป็นก้อน เรียกใช้ซ้ำได้

- เหมือนสูตรอาหาร — เขียนขั้นตอนไว้ครั้งเดียว ทำกี่ครั้งก็ได้
- รวมกลุ่มคำสั่งไว้ในชื่อเดียว เรียกใช้ซ้ำได้โดยไม่ต้องเขียนใหม่
- รับค่าเข้า (**parameter**) และคืนค่าออก (**return**) ได้
- ช่วยให้โค้ดอ่านง่ายขึ้น ไม่ซ้ำซ้อน — พื้นฐานก่อนเขียนโปรแกรมใหญ่ในวันที่ 2

```
functions.py

def greet(name):

    return "สวัสดี " + name

print(greet("Got"))

# สวัสดี Got
```

รู้จัก Pi Camera Module 3

- กล้องทางการของ Raspberry Pi ความละเอียด **12MP** เซนเซอร์ Sony IMX708
- รองรับ **Autofocus (PDAF)** โฟกัสอัตโนมัติได้ ไม่ต้องหมุนเลนส์เอง
- เชื่อมต่อผ่านพอร์ต **CSI (Camera Serial Interface)** บนบอร์ด Pi5 โดยเฉพาะ
- มีเวอร์ชัน **Standard** และ **Wide FOV** ให้เลือกตามมุมมองที่ต้องการ



[ใส่ภาพ Pi Camera Module 3 จริง]

ไม่มีภาพ commons ที่ชัดเจน — แนะนำถ่ายภาพสินค้าเอง
หรือดึงจาก raspberrypi.com

ต่อสาย CSI เข้าบอร์ด

- ปิดเครื่อง Pi5 ก่อนต่อสายทุกครั้ง เพื่อความปลอดภัยของอุปกรณ์
- ดึงคลิปล็อกที่พอร์ต CSI ขึ้นเบา ๆ ก่อนเสียบสาย
- เสียบสายให้ด้านโลหะ (contact) หันเข้าหาพอร์ต HDMI — ต่อผิดด้านจะไม่ติด
- กดคลิปล็อกกลับให้แน่น แล้วค่อยเปิดเครื่อง

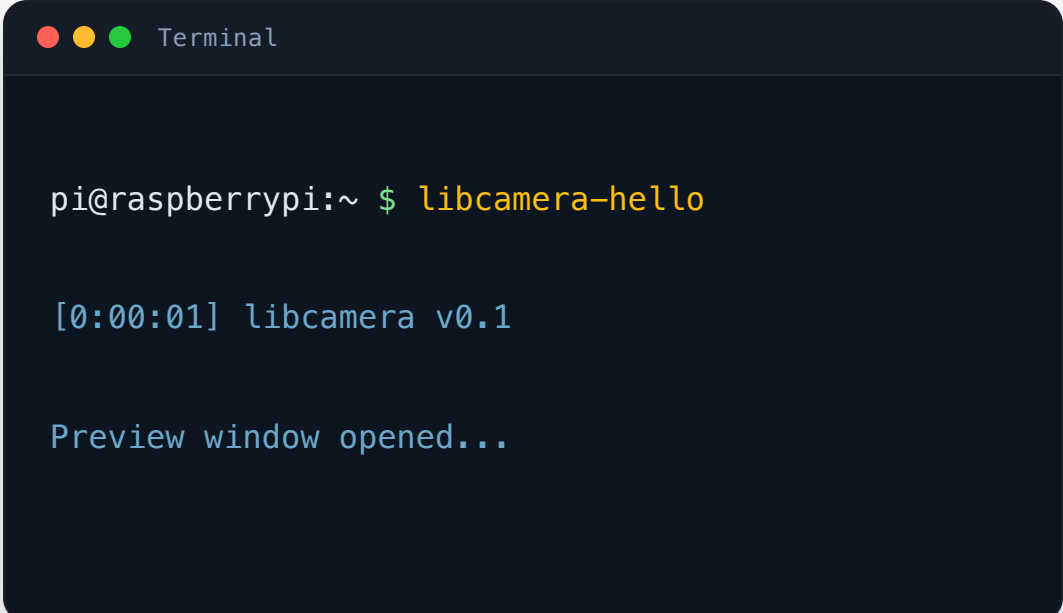


[ใส่ภาพขั้นตอนต่อสาย CSI]

แนะนำถ่ายภาพ step-by-step จริงหน้างานแทน mockup

เปิดใช้งานกล้องครั้งแรก

- เปิด Terminal แล้วทดสอบด้วยคำสั่ง **libcamera-hello** เพื่อเช็คว่ากล้องต่อติด
- ถ้าเห็นภาพพรีวิวขึ้นมา แปลว่ากล้องพร้อมใช้งานแล้ว
- ถ้าไม่เจอกล้อง ให้เช็กทิศทางสายอีกครั้ง แล้วรีสตาร์ทเครื่อง

A terminal window titled "Terminal" with three colored window control buttons (red, yellow, green) on the top left. The terminal shows the command `pi@raspberrypi:~ $ libcamera-hello` being executed. The output consists of two lines: `[0:00:01] libcamera v0.1` and `Preview window opened...`.

```
Terminal

pi@raspberrypi:~ $ libcamera-hello

[0:00:01] libcamera v0.1

Preview window opened...
```

ถ่ายภาพนิ่งด้วย picamera2

- **picamera2** คือไลบรารี Python อย่างเป็นทางการสำหรับควบคุมกล้อง Pi
- สร้าง object กล้อง เริ่มการทำงาน แล้วสั่งถ่ายภาพด้วยคำสั่งเดียว
- ภาพที่ได้จะถูกบันทึกเป็นไฟล์ทันที พร้อมนำไปใช้ในขั้นต่อไป

```
capture.py

from picamera2 import Picamera2

cam = Picamera2()

cam.start()

cam.capture_file("photo.jpg")
```

บันทึกวิดีโอ + รีวิวสด

- ใช้ไลบรารีเดียวกับตอนถ่ายภาพ แต่เปลี่ยนเป็นโหมดบันทึกวิดีโอ
- **start_preview()** แสดงภาพสดจากกล้องบนหน้าจอ ใช้เช็กมุมมองกล้องก่อนถ่ายจริง
- เอา Python ที่เพิ่งเรียนมาใช้จริงทันที — เขียนสคริปต์ถ่ายภาพ/วิดีโอเองได้ทั้งหมด

```
record.py

from picamera2 import Picamera2

cam = Picamera2()

cam.start_preview()

cam.start_and_record_video("clip.mp4", duration=5)
```

Python + OpenCV เบื้องต้น

- ติดตั้งไลบรารี OpenCV โหลด/แสดงภาพด้วยโค้ดไม่กี่บรรทัด
- **Grayscale** — ลดข้อมูลสีเพื่อประมวลผลเร็วขึ้น
- **Resize / Crop** — ปรับขนาดและตัดเฉพาะส่วนที่สนใจ
- **Edge detection (Canny)** และ **Threshold** — หาขอบวัตถุ แยกพื้นหน้า-หลัง



Preview: หา "กรอบป้ายทะเบียน" ด้วยมือ + สรุปรวัน 1

- ลอง **contour detection** หาเส้นขอบสี่เหลี่ยมป้ายทะเบียนบนภาพตัวอย่าง
- เห็นด้วยตาว่าวิธีนี้ไวต่อแสง มุมกล้อง และพื้นหลังที่ซับซ้อน
- สรุปสิ่งที่เรียนมาทั้งวัน: RPi, Python, OpenCV เบื้องต้น
- ปูทางคำถาม: ถ้าวิธีคลาสสิกไม่พอ พรุ่งนี้จะใช้อะไรมาช่วย?



Contour บนภาพตัวอย่าง

ใส่ภาพก่อน/หลังทำ contour detection บนภาพรถ
ตัวอย่าง

วันที่ 2



YOLOv8n + OCR ประกอบระบบอ่านป้ายทะเบียนจริง

🎯 ใช้ AI จริงแก้ข้อจำกัด → ประกอบระบบจบครบ

บทวน Day 1 + จุดอ่อนของ contour method

- บทวนเส้นทางเมื่อวาน: RPi → Python → OpenCV → Contour detection
- จุดอ่อนที่เจอ: แสงเปลี่ยน มุมป้ายเอียง พื้นหลังรก ทำให้ตรวจจับพลาด
- คำถามเปิดห้อง: "ถ้ากฎเกณฑ์ตายตัวไม่พอ จะให้คอมพิวเตอร์เรียนรู้เองได้ไหม?"

Contour: แสงเปลี่ยน ✖



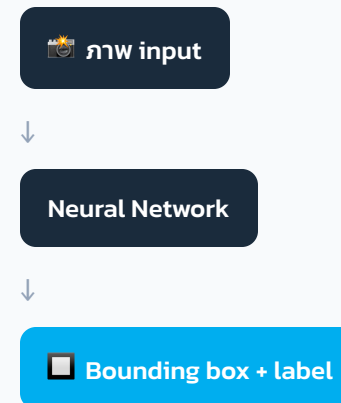
Contour: มุมเอียง ✖

↓ แล้วจะแก้ยังไง?

AI Model จริง →

ความรู้เสริม: YOLOv8 คืออะไร

- **Object detection** คือการให้โมเดลบอกว่าในภาพมีอะไร และอยู่ตรงไหน (bounding box)
- โมเดลเรียนรู้จากข้อมูลที่มนุษย์ annotate ไว้ ด้วยเครื่องมืออย่าง **Roboflow**
- ขั้นตอน train ทำบน **Colab** ซึ่งมี GPU ให้ใช้ฟรี
- คอร์สนี้ใช้โมเดลที่ train ไว้แล้ว (pretrained) — เข้าใจหลักการ ไม่ต้อง train เอง



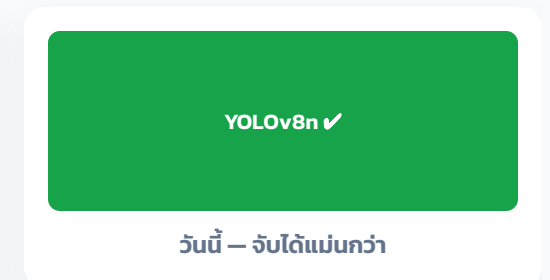
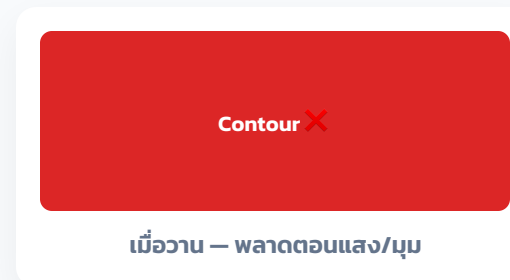
ตรวจสอบ/เข้าใจ environment ระบบ Edge-AI

- โครงสร้างโฟลเดอร์โปรเจกต์ **LicensePlate-EdgeAI** แบ่งเป็นส่วน model และสคริปต์
- โหลด **pretrained model 2 ตัว**: ตัวตรวจจับป้าย และตัวช่วยอ่านข้อความ
- หน้าทีแต่ละไลบรารี: **ultralytics** (YOLO), **opencv** (ภาพ), **pytesseract** (OCR), **picamera2** (กล้อง)

```
LicensePlate-EdgeAI/  
├── models/  
│   ├── yolov8n_plate.pt  
│   └── ocr_config.json  
├── detect.py  
├── ocr_reader.py  
└── main.py
```

รัน YOLOv8n ตรวจจับป้ายทะเบียนจริง

- รันโมเดล YOLOv8n กับภาพนิ่งและกล้องสด
- ดูค่า confidence และกรอบที่โมเดลตรวจจับได้
- เทียบผลลัพธ์กับ **contour method** เมื่อวาน — เห็นความแม่นยำที่ต่างกันชัดเจน



OCR ด้วย Tesseract (รองรับไทย)

- กรอบภาพเฉพาะส่วนป้ายที่ YOLO ตรวจจับได้ ก่อนส่งเข้า OCR
- **Tesseract** อ่านตัวอักษรจากภาพ รองรับทั้งภาษาไทยและอังกฤษ
- **ทำความสะอาดข้อมูล (data cleaning):** ตัดอักขระแปลกปลอม แยกเลขทะเบียนกับจังหวัด



ocr_output.txt

Raw OCR:

"1กก 1234 | กรุงเทพฯ"

After cleaning:

plate: "1กก1234"

province: "กรุงเทพฯ"

ประกอบ Pipeline เต็มระบบ



ต่อทุกส่วนที่เรียนมาเข้าด้วยกัน: กล้อง → YOLO detect → crop → OCR → เทียบ whitelist → แสดงผล

เริ่มจากรันผ่าน **terminal/GUI** ง่าย ๆ ก่อน ค่อยเพิ่มความสวยงามทีหลัง

ทดสอบทีละจุดในระบบ ก่อนรวมเป็น pipeline เดียว

ทดสอบกับรถจริง/ภาพจริง

- แบ่งกลุ่มทดสอบระบบกับรถจริงหรือภาพที่เตรียมมา
- สังเกตปัญหาหน้างาน: แสง มุมกล้อง ระยะห่าง ความคมชัด
- ช่วยกันปรับพารามิเตอร์ในระบบเพื่อให้ผลลัพธ์แม่นยำขึ้น



ภาพบรรยากาศทดสอบกลุ่ม

แนะนำใช้ภาพกิจกรรมจริงจาก workshop ครั้งก่อนของ
Cytron

นำเสนอผลงาน + Q&A + ปิดคอร์ส

- แต่ละกลุ่มโชว์ระบบตัวเอง พร้อมเล่าปัญหาที่เจอและวิธีแก้
- เปิด Q&A ให้ถามคำถามที่ค้างใจตลอด 2 วัน
- 🎓 แจก Certificate ปิดคอร์ส



ขอบคุณที่ร่วม Workshop